

Ejercicio 14

Dado el siguiente algoritmo:

```
public static int recursivo( int n ) {  
    if (n <= 1){  
        return 1;  
    }  
    int i = 1;  
    while (i <= n) {  
        i*=2;  
    }  
    return n + recursivo(n/2) * n/2;  
}
```

Calcular la función de tiempo de ejecución y el orden de ejecución de la misma.

- Primero tenemos el caso base que es una operación constante y representamos con c_1 .
- Después tenemos una iteración que se ejecuta desde 1 hasta n , inclusive. En cada iteración, el índice aumenta multiplicándose por 2 en cada vuelta. Cambio del índice:

1. $i = 1$
2. $i = 1 \cdot 2$
3. $i = 1 \cdot 2 \cdot 2 = 2^2$
4. $i = 1 \cdot 2 \cdot 2 \cdot 2 = 2^3$

Término general: $k. \quad i = 2^{k-1}$

Como la sumatoria aumenta de uno en uno, hay que encontrar el límite superior de la sumatoria que se ajuste al último valor que va a tomar el índice.

Despejamos i .

$$\begin{aligned}n &= i \\2^{k-1} &= i \\k - 1 &= \log_2(i) \\k &= 1 + \log_2(i)\end{aligned}$$

Cada iteración se ejecuta $1 + \log_2 n$ veces. La sumatoria queda

$$\sum_{i=1}^{1+\log_2 n} (c_2)$$

- Por último, tenemos una llamada recursiva que envía como parámetro $\frac{n}{2}$. La sumatoria queda,

$$T(n) = \begin{cases} c_1 & n \leq 1 \\ c_2 + \sum_{i=1}^{1+\log_2 n} c_2 + T(\frac{n}{2}) & n > 1 \end{cases}$$

Para facilitar el desarrollo de la recursión, utilizando esta propiedad

$$\sum_{k=m}^n c = c \cdot (n - m + 1)$$

podemos reescribir el segundo término,

$$\sum_{i=1}^{1+\log_2 n} c_2 = c_3(1 + \log_2 n - 1 + 1) = c_3(1 + \log_2 n)$$

Desarrollo de la recursión

1. $T(n) = c_2 + c_3(1 + \log_2 n) + T(\frac{n}{2})$
2. $T(n) = c_2 + c_3(1 + \log_2 n) + (c_2 + c_3(1 + \log_2 n) + T(\frac{n}{2^2})) = 2c_2 + 2c_3(\log_2 n) + T(\frac{n}{2^2})$
3. $T(n) = 2c_2 + 2c_3(\log_2 n) + (c_2 + c_3(1 + \log_2 n) + T(\frac{n}{2^2 \cdot 2})) = 3c_2 + 3c_3(1 + \log_2 n) + T(\frac{n}{2^3})$

Paso i : $T(n) = i \cdot c_2 + i \cdot c_3(1 + \log_2 n) + T(\frac{n}{2^i})$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} 1 &= \frac{n}{2^i} \\ 2^i &= n \\ i &= \log_2 n \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general. Propiedad a usar:

- $a^{\log_a(b)} = b$

$$\begin{aligned}
T(n) &= c_2 \log_2 n + c_3 \log_2 n(1 + \log_2 n) + T\left(\frac{n}{2^{\log_2 n}}\right) \\
&= c_2 \log_2 n + c_3 \log_2 n + c_2 \log_2^2 n + T(1) \\
&= \log_2 n \cdot (c_2 + c_3) + c_3 \log_2^2 n + c_1
\end{aligned}$$

Renombramos $c_4 = c_2 + c_3$

$$= \log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1$$

Entonces, la función de tiempo de ejecución del método es

$$T(n) = \log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = \log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1$$

Hay que buscar los valores constantes c y n_0 tales que

$$\log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1, \text{ para todo } n \geq n_0$$

- Primer término, $\log_2 n \cdot c_4$
Sabemos que $\log_2 n \leq \log_2^2 n$.
Si multiplicamos a ambos lados de la desigualdad por c_4 .

$$c_4 \log_2 n \leq c_4 \log_2^2 n$$

Encontramos un valor k_1 que acota al término. Si tomamos $n_0 = 1$.

$$\begin{aligned}
c_4 \log_2 1 &\leq c_4 \log_2^2 1 \\
c_4 \cdot 0 &\leq c_4 \cdot 0 \\
0 &\leq 0
\end{aligned}$$

Tenemos el $k_1 = c_4$ y $n_0 = 1$ que acotan el término.

- Segundo término, $c_3 \log_2^2 n$
Sabemos que $\log_2^2 n \leq \log_2^2 n$.
Si multiplicamos a ambos lados de la desigualdad por c_3 .

$$c_3 \log_2^2 n \leq c_3 \log_2^2 n$$

Encontramos un valor k_2 que acota al término. Si tomamos $n_0 = 1$.

$$\begin{aligned}
c_3 \log_2^2 1 &\leq c_3 \log_2^2 1 \\
c_3 \cdot 0 &\leq c_3 \cdot 0 \\
0 &\leq 0
\end{aligned}$$

Tenemos el $k_2 = c_3$ y $n_0 = 1$ que acotan el término.

- Tercer término, c_1
Sabemos que $1 \leq \log_2^2 n$.
Si multiplicamos a ambos lados de la desigualdad por c_1 .

$$c_1 \leq c_1 \log_2^2 n$$

Encontramos un valor k_3 que acota al término. Si tomamos $n_0 = 2$.

$$\begin{aligned} c_1 \log_2 2 &\leq c_1 \log_2^2 2 \\ c_1 \cdot 1 &\leq c_1 \cdot 1 \\ c_1 &\leq c_1 \end{aligned}$$

Tenemos el $k_1 = c_1$ y $n_0 = 2$ que acotan el término.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1 \leq k_1 \log_2^2 n + k_2 \log_2^2 n + k_3 \log_2^2 n$$

$$T(n) \leq (k_1 + k_2 + k_3) \log_2^2 n$$

$$T(n) \leq (c_4 + c_3 + c_1) \log_2^2 n$$

Renombramos $c = c_4 + c_3 + c_1$

$$T(n) \leq c \cdot \log_2^2 n$$

El n_0 más restrictivo es $n_0 = 2$ porque cumple con todos los términos para $n_0 \geq 2$. Por lo tanto,

$$T(n) \leq O(\log_2^2 n), \text{ con } c = c_4 + c_3 + c_1 \text{ para todo } n \geq n_0, \text{ con } n_0 = 2$$